

AssetFlow

Smart Asset Management & Resource Allocation Platform

Design Document · MERN Stack · IIT Roorkee Challenge

#	Team Member	Enrollment No.
01	Mahendra Limanpure	23118043
02	Harshit Kumar	23118034

Competition	Smart Asset Management Challenge — IIT Roorkee	Stack	MongoDB · Express · React · Node.js
Repository	github.com/mahendra07limanpure/AssetFlow	Date	June 2026

1

Problem Understanding

The **Cultural Council of IIT Roorkee** manages a large shared pool of resources — DSLR cameras, studio lighting, audio systems, costumes, stage props, and recording equipment — frequently requested across multiple sections and events. Without a centralized system, coordination relies on fragmented communication, spreadsheets, and informal registers, resulting in scheduling conflicts, asset underutilization, and untracked returns.

AssetFlow solves this by providing a centralized digital platform where administrators control inventory and approve requests, while users can browse, book, and track assets through a structured role-based approval workflow — eliminating all manual coordination.

Core Challenges Addressed

- No real-time inventory visibility
- Manual and error-prone booking
- No standardized approval workflow
- Missing utilization analytics
- No return deadline enforcement
- No audit trail for movements
- Overdue assets not flagged
- Borrowing history fragmented

2

System Architecture

AssetFlow follows a **three-tier MERN architecture**: a React.js SPA on the client tier, a RESTful Express/Node.js API in the logic tier, and MongoDB Atlas in the persistence tier. JWT tokens secure every route; role-based middleware enforces admin vs. user separation.

Layer	Technology	Responsibility
Frontend	React.js 18, Tailwind CSS, Recharts	SPA rendering, routing, state, data visualization
Backend	Node.js 18, Express.js v4	REST API, business logic, JWT auth, role middleware
Database	MongoDB Atlas, Mongoose ODM	Document storage, schema validation, indexing
Auth	JSON Web Tokens, bcrypt	Stateless auth, password hashing, session management
QR Module	qrcode (npm)	Server-side QR generation per asset for scan operations

Five primary Mongoose collections. Relationships use ObjectId references — no joins required.

users Collection

Field	Type	Description
_id	ObjectId	Auto-generated primary key
name	String	Full name
email	String	Unique (indexed)
password	String	bcrypt-hashed
role	String	'admin' 'user'
createdAt	Date	Account timestamp

assets Collection

Field	Type	Description
_id	ObjectId	Primary key
name	String	e.g., DSLR Camera
category	String	e.g., Photography
description	String	Detailed description
totalQuantity	Number	Total units owned
availableQuantity	Number	Currently available
status	String	'available' 'maintenance' 'retired'
condition	String	'good' 'fair' 'damaged'
qrCode	String	Base64 QR (optional)
createdBy	ObjectId	Ref: users

bookings Collection

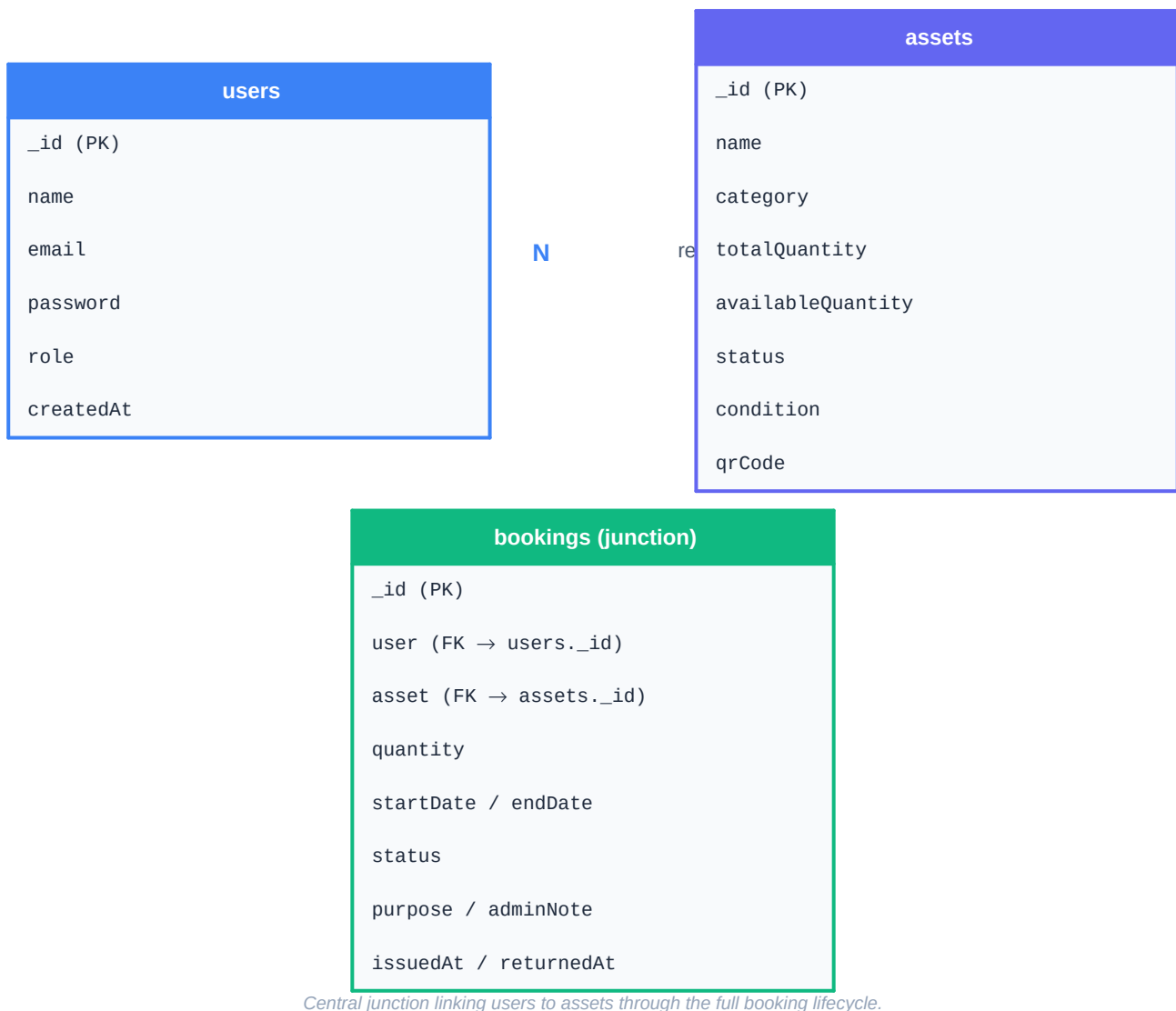
Field	Type	Description
_id	ObjectId	Primary key
user	ObjectId	Ref: users — requester
asset	ObjectId	Ref: assets — item
quantity	Number	Units requested
startDate	Date	Borrow start
endDate	Date	Expected return

status	String	'pending' 'approved' 'rejected' 'issued' 'returned'
purpose	String	Reason for request
adminNote	String	Admin note (optional)
issuedAt	Date	Actual issue time
returnedAt	Date	Actual return time

auditlogs Collection		
Field	Type	Description
_id	ObjectId	Primary key
action	String	e.g., BOOKING_APPROVED
performedBy	ObjectId	Ref: users — actor
targetId	ObjectId	Affected resource ID
targetType	String	'Asset' 'Booking' 'User'
details	String	Human-readable description
timestamp	Date	When it occurred

4

Entity Relationship Diagram



5

API Overview

Authentication Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user account	Public
POST	/api/auth/login	Login, receive JWT	Public
GET	/api/auth/me	Get current user profile	JWT

Asset Endpoints

Method	Endpoint	Description	Auth
--------	----------	-------------	------

GET	/api/assets	List all assets (filters)	JWT
POST	/api/assets	Create new asset	Admin
GET	/api/assets/:id	Get asset details	JWT
PUT	/api/assets/:id	Update asset info	Admin
DELETE	/api/assets/:id	Delete asset	Admin
GET	/api/assets/:id/qr	Generate asset QR code	Admin

Booking Endpoints

Method	Endpoint	Description	Auth
GET	/api/bookings	List bookings (role-scoped)	JWT
POST	/api/bookings	Create booking request	User
PUT	/api/bookings/:id/approve	Approve pending request	Admin
PUT	/api/bookings/:id/reject	Reject pending request	Admin
PUT	/api/bookings/:id/issue	Mark asset as issued	Admin
PUT	/api/bookings/:id/return	Mark asset as returned	Admin

6

Design Decisions

MERN Stack

MongoDB's document model maps naturally to assets with variable sub-fields. Express/Node provides a lightweight API layer while React enables a responsive SPA without full-page reloads.

JWT Stateless Auth

JWTs allow horizontal scaling with no shared session storage. Tokens carry the user role, enabling middleware role checks without extra DB round-trips.

Availability Guard

Before a booking is created, the API validates `asset.availableQuantity ≥ requested quantity` (400 otherwise). On issue/return, `availableQuantity` is atomically updated.

Role Middleware

A reusable 'protect' middleware verifies the JWT; a separate 'adminOnly' middleware checks the role. Clean separation means any route is secured with a single line.

Audit Log Collection

Every significant state change writes to auditlogs with actor, target, and timestamp — providing a tamper-evident trail without polluting core collections.

Soft Quantity Reserve

Units are decremented only on 'issued' (not on approval), preventing phantom reservations and giving admins discretion during the approval window.

7

Feature Completeness

Feature	Status	Notes
User Authentication (JWT)	Implemented	Register, login, role-based access
Inventory Management CRUD	Implemented	Admin add/edit/delete/categorize
Asset Discovery & Search/Filter	Implemented	Browse, filter by category/status
Booking Request System	Implemented	Date range, quantity, purpose
Approval / Rejection Workflow	Implemented	Admin review with notes
Asset Issue & Return Tracking	Implemented	Inventory auto-updated each state
Analytics Dashboard	Implemented	Bar, Pie, Line charts

Borrowing History	Implemented	Full lifecycle per user/system
QR Code Generation	Implemented	Per-asset QR for scan ops
Audit Logs	Implemented	All major actions logged
Asset Health Tracking	Implemented	Condition field + maintenance notes
In-App Notifications	Implemented	Status updates in UI
Dockerized Deployment	Planned	Docker Compose config in repo

8 Technology Stack & Scalability

Component	Technology	Notes
Frontend	React.js 18 + Tailwind CSS	Component SPA, responsive, utility styling
Charts	Recharts	Bar, Pie, Line for dashboard
HTTP Client	Axios	Interceptors for JWT attachment
Backend	Node.js 18 + Express.js v4	RESTful routing, middleware chain
Database	MongoDB Atlas + Mongoose	Cloud-hosted, ODM schema validation
Auth	JWT + bcryptjs	Stateless tokens, hashed passwords
QR	qrcode (npm)	Server-side SVG/PNG generation
Config	dotenv	Environment-driven secrets

■ MongoDB Indexing	Email, category, and status fields are indexed for fast queries at scale.
■ Stateless JWT API	Horizontal scaling behind a load balancer — no sticky sessions needed.
■ React Code Splitting	Lazy-loaded routes reduce bundle size and keep UI fast as features grow.
■ Soft-Delete Pattern	Assets/bookings use status fields (never hard-deleted), preserving analytics.
■ Env-Driven Config	All secrets via env vars enables seamless dev → staging → prod promotion.